Joint Optimization of Resource and Pricing for Collaborative CNN Inference in Edge Computing

Maowen Li, Xiaolong Xu, *Senior Member, IEEE*, Guangming Cui, Yong Cheng, Fei Dai, Wanchun Dou, Lianyong Qi, *Senior Member, IEEE*, and Zhipeng Cai, *Fellow, IEEE*

***Abstract*—Convolutional Neural Networks (CNN) have been widely adopted in multi-access edge computing systems due to their powerful feature extraction and high inference accuracy. Harnessing the heterogeneous computational capacities of end devices and edge servers, end-edge collaborative inference addresses resource constraints of end devices and latency challenges of real-time inference. However, existing studies typically assume that all potential inference models are pre-employed on the server, which may not always align with real-world deployment scenarios. Moreover, the economic incentives of edge service providers are often overlooked in current distributed inference research. To address these issues, we propose an end-edge collaborative inference framework for CNN in multi-access edge computing systems, named DisCNN. We first model the interaction between latency-sensitive end devices (EDs) and resource-constrained edge service providers as a Stackelberg game and prove the existence of a Subgame Perfect Equilibrium. Next, we derive the optimal response characteristics of the EDs and develop an efficient algorithm to allocate computation and communication resources to EDs while partitioning the CNN accordingly. To compute the optimal offloading set, we introduce a linear-time algorithm with a bounded approximation ratio. Then, to jointly optimize caching, resource allocation, and pricing, we design an efficient approximate algorithm to compute a near-optimal solution. Finally, extensive simulation results validate the effectiveness of the proposed framework.**

***Index Terms*—End-edge collaborative inference, Stackelberg game, multi-access edge computing, resource management and pricing.**

Received 7 January 2025; revised 19 October 2025; accepted 28 October 2025. Date of publication 30 October 2025; date of current version 6 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62372242 and Grant 92267104, in part by the Jiangsu Provincial Major Project on Basic Research of Cutting-edge and Leading Technologies, under Grant BK20232032, and in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 24KJB520021. Recommended for acceptance by X. Wang. *(Corresponding author: Xiaolong Xu.)*

Maowen Li and Yong Cheng are with the School of Software, Nanjing University of Information Science & Technology, Nanjing 210044, China (e-mail: maowenlinuist@gmail.com; yongcheng@nuist.edu.cn).

Xiaolong Xu and Guangming Cui are with the School of Software, Nanjing University of Information Science & Technology, Nanjing 210044, China, and also with the Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services, and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: xlxu@ieee.org; gcui@nuist.edu.cn).

Fei Dai is with the School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming 650224, China (e-mail: flydai.cn@gmail.com).

Wanchun Dou is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: douwc@nju.edu.cn).

Lianyong Qi is with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266024, China (e-mail: lianyongqi@gmail.com).

Zhipeng Cai is with the Department of Computer Science, Georgia State University, Atlanta, GA 30302 USA (e-mail: zcai@gsu.edu).

Digital Object Identifier 10.1109/TMC.2025.3627519

I. Introduction

In RECENT years, the rapid development of smart end devices has driven the widespread deployment of devices with diverse capabilities, including image recognition, video analysis, and sensor data processing [1], [2], [3]. Examples include smart cameras, wearable gadgets, intelligent transportation systems, and industrial automation systems [4], [5]. These devices generate large volumes of data for real-time analysis and decision-making. Efficient data processing and analysis, increasingly reliant on edge computing, have become crucial for enhancing the functionality and intelligence of these smart end devices [6], [7].

Convolutional Neural Networks (CNN), as a prominent model in deep learning, have demonstrated superior performance in image recognition, speech processing, and natural language understanding, supporting data processing and intelligent decision-making across multi-access edge computing systems [8]. Despite these advantages, the computational complexity and large parameter size of CNN models impose significant memory and processing constraints for deployment on resource-constrained end devices (EDs), which often have limited computing power even with GPUs [9]. The traditional cloud-centric architecture offers a potential solution. However, its applicability is limited in real-time scenarios due to issues such as data transmission latency, privacy concerns, and bandwidth constraints.

Mobile Edge Computing (MEC) enables computation offloading from remote clouds to the edge, reducing network traffic and enhancing computational capacity near end devices [10]. This capability facilitates distributed inference of CNN by partitioning and offloading computation across edge devices and servers, a topic that has been extensively studied in edge computing environments [11], [12], [13], [14], [15]. However, most existing studies assume that all potential inference models are pre-deployed on edge servers, an assumption that overlooks the constrained storage capacity of edge servers, which is often fixed and non-expandable. Moreover, existing research predominantly focuses on optimizing user-centric metrics, such as latency and energy consumption [16], [17], [18], [19], while the economic incentives and operational objectives of edge service providers remain largely unexplored. From the service provider's perspective, balancing limited storage, computation,

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

and communication resources while meeting user latency requirements remains a critical yet underexplored challenge.

In fact, computation, communication, storage, and pricing are interdependent factors in collaborative inference of CNN models within multi-access edge computing systems. Specifically, the caching status of a model on the edge server determines whether an ED can participate in collaborative inference. The allocation of computation and communication resources dictates whether the offloading strategy meets offloading requirements, while pricing influences the ED's offloading decisions. These factors jointly impact the utility of the service provider, who aims to maximize its profit while managing limited resources. However, the constraints on EDs' latency and energy consumption introduce conflicting objectives, making collaborative inference a complex challenge in multi-access edge computing systems. Unlike prior studies that apply game theory to resource allocation in wireless networks, focusing primarily on latency or energy efficiency [20], [21], or explore economic and capacity constraints without addressing their interplay [22], [23], our framework provides a comprehensive solution for CNN-based collaborative inference by optimizing the interaction of storage, computation, communication, and pricing in edge computing systems.

In this work, we propose an end-edge collaborative inference framework for CNN in multi-access edge computing systems, named DisCNN, addressing the aforementioned issues from an economic perspective. Specifically, we model the interaction between a utility-maximizing edge service provider and cost-minimizing EDs as a Stackelberg game. The edge service provider optimizes its caching, resource allocation, and pricing strategies, which are announced to the EDs. The EDs then respond by making decisions that minimize their own costs based on these strategies. Based on this framework, we design corresponding resource allocation and pricing schemes that maximize the service provider's utility while ensuring that EDs make energy-optimal decisions that meet their latency requirements. Main contributions of this paper are shown as follows:

- We model the interaction between latency-sensitive EDs and resource-constrained edge service providers as a Stackelberg game and prove the existence of Subgame Perfect Equilibrium in this game.
- We theoretically derive the optimal response characteristics of the EDs and design an efficient algorithm to allocate computation and communication resources to EDs and partition the CNN model.
- We design a lightweight linear-time algorithm with a bounded approximation ratio to efficiently select offloading devices for collaborative inference, ensuring effective utilization of computational resources and satisfying latency constraints.
- We develop an efficient approximate algorithm to determine the optimal set of models to cache on edge servers, enabling efficient model access while maximizing resource utilization under limited storage constraints.
- The effectiveness of the proposed framework is confirmed through extensive experiments.

II. Related Work

In this section, we first introduce related work on distributed inference empowered by edge resources, and then we review the extensive research on resource allocation within the framework of Stackelberg games.

A. Collaborative Inference

With the development of deep neural networks, an increasing number of models with complex structures and significant computational demands have emerged. Traditional architectures are cloud-centric, relying on the powerful computing and storage resources of cloud servers for task learning and inference. However, with the advent of intelligent end devices generating massive amounts of data, the latency caused by transmitting data to the cloud has become intolerable for latency-sensitive tasks. Furthermore, due to the limited resources of end devices, local computation often struggles to maintain service quality. Therefore, distributed inference leveraging edge computing has become a promising solution to address these challenges.

Liu et al. [24] designed an edge-cloud collaborative inference system that optimizes resource utilization across endpoint devices, edge servers, and the cloud by partitioning deep neural networks (DNN) into three components using model compression and segmentation. Additionally, the system reduces the amount of data transmitted between devices through model compression, further decreasing inference latency. Yang et al. [25] proposed a distributed collaborative inference framework called OfpCNN, which achieves accurate prediction of CNN layer computation delays using a latency prediction model based on floating-point operations and CPU load (FCPM). They employ a hybrid vertical-horizontal partitioning method (HVPM) to finely segment CNN across heterogeneous devices, effectively enhancing the model's inference speed. Chen et al. [26] proposed a distributed edge computing framework that integrates edge computing and parallel computing to accelerate DNN inference tasks on edge devices. Addressing the Minimum Latency joint Communication and Computation Scheduling (MLCCS) problem, the study overcomes the limitations of existing research, which assumes fixed communication times and arbitrarily divisible workloads, and develops a more generalized and practical model. Chen et al. [27] investigated an aerial edge computing system based on high-altitude platforms and drones, aimed at reducing the energy consumption of ground devices through task offloading and resource allocation. To address dynamic task arrival and wireless communication quality issues, they proposed a distributed online task offloading and resource allocation algorithm, integrating convex optimization and game theory to design multi-server selection and power allocation algorithms.

The aforementioned studies primarily focus on reducing latency and system energy consumption in distributed inference scenarios, while often overlooking the model storage requirements and the utility of service providers. In edge networks, the limited and non-expandable storage resources of edge servers make storage issues crucial. Furthermore, in resource-constrained environments, it is essential to consider appropriate

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

incentive mechanisms to ensure the economic utility of edge service providers. To address these challenges, we jointly optimize storage, computation, communication, and pricing, ensuring that user service quality is maintained while safeguarding the interests of edge service providers.

B. Stackelberg Game

The Stackelberg game is an asymmetric game model that describes the strategic interactions between leaders and followers based on complete information. Its key characteristic is that the leader can anticipate the followers' reactions and formulate optimal strategies accordingly, thereby optimizing the efficiency and fairness of resource allocation. As a result, it has been widely applied in various resource allocation scenarios.

Shao et al. [28] proposed a non-cooperative multi-defender Stackelberg game model for resource allocation, introducing a risk-sharing mechanism to address the anarchic phenomenon in decentralized systems by combining individual and global optimal strategies. Huang et al. [29] developed a single-leader multiple-follower Stackelberg game model to optimize service provision in the Metaverse, focusing on users' retention of offloading services while minimizing redundant data transmission and monetary costs. Xie et al. [30] utilized a multi-leader-multi-follower Stackelberg game model to design a differentiated pricing strategy for bandwidth allocation, enhancing resilience against link flooding attacks while incentivizing resource deployment by internet service providers. Tong et al. [22] introduced a Stackelberg game model to address pricing and task offloading in mobile edge computing, proposing unified and differentiated pricing algorithms to enhance server profits and reduce latency for end users under computing capacity constraints. Similarly, Datar et al. [23] modeled a 5G network slicing scenario as a Stackelberg game, where application service providers compete for virtualized network resources. They proposed two market mechanisms: a trading post mechanism for resource allocation with pre-assigned budgets and a pricing game for unconstrained budgets, deriving market prices as duals of resource capacity constraints and proving the existence of a unique variational equilibrium.

While these studies demonstrate the efficacy of Stackelberg games in optimizing resource allocation, latency, and energy consumption in wireless and mobile networks, they often overlook the specific storage constraints and economic objectives critical to edge service providers in CNN-based collaborative inference. In contrast, our work leverages a Stackelberg game framework to jointly optimize computation, communication, storage, and pricing, enabling efficient resource allocation and economic incentives for edge-based CNN inference, effectively addressing the demands of resource-constrained edge servers and latency-sensitive end devices.

III. System Model

In this section, the system model is first introduced. Then, the process of EDs performing local inference tasks and engaging in end-edge collaborative inference is discussed in detail.

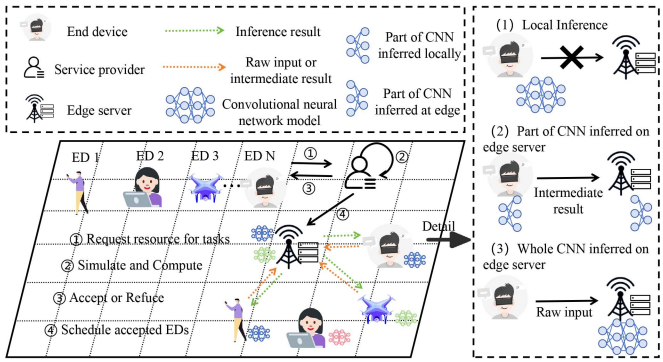


Fig. 1. The system model of collaborative inference for CNN.

TABLE I
Main Symbols Used in the Proposed Model

Symbol	Description
$N = \{e_1, e_2, \ldots, e_n\}$	End devices
S	Storage capacity of the edge server
M_i	CNN model
J	Set of CNN models
X	Cached models on the edge server
L_i	Number of model layers
S_i	Model's memory footprint
F_i	Computation needed per layer
O_i	Output data per layer
π_i	Fees paid by ED_i
r_i^l	Local inference time of ED_i
t_i	The task deadlines of ED_i
f_i^l	Local resources of ED_i
a_i	Offloading decision of ED_i
a_{-i}	Set of offloading decisions of end devices except ED_i
d_i	Partition layer of the model
f_i	Computation allocated by the edge server
w_i	Bandwidth allocated by the edge server
p_i	Transmission power of ED_i
h_i	Channel coefficient of ED_i
I_c	Average co-channel interference power
σ_i^2	Noise at the edge server
k_i^l	Energy consumption parameter
γ_i	The unit local energy cost
β_i	Transmission power efficiency parameter
ρ_i	The resources allocated to ED_i
$r_i^l(d_i)$	Local inference time in collaboration
$r_i^e(d_i)$	Edge inference time in collaboration
$r_i^u(d_i, w_i)$	Transmission time of ED_i
$T_i^1(d_i)$	FLOPs for local inference
$T_i^2(d_i)$	FLOPs for edge inference
C_i^0	Local inference cost
$C_i^1(d_i, \rho_i, a_{-i})$	Collaborative inference cost

A. MEC Model

We consider a multi-access edge computing system which consists of an edge service provider with an edge server and a set $N = e_1, e_2, \ldots, e_n$ of EDs, as shown in Fig. 1, and the main symbols used in the proposed model are listed in Table I. n represents the total number of EDs. The edge server is managed by a service provider and has a storage capacity of S. If an end device $ED_i \in N$ wishes to execute a model $M_i \in J$, where J represents a set of CNN models, the corresponding model must be cached on the edge server before inference can take place. The inference model of ED_i is characterized by the following aspects: the number of layers of the model L_i, the model's memory footprint S_i, a list of computational requirements for each layer F_i, a list of output data sizes for each layer O_i. The service provider decides to cache a subset $X \subseteq J$ subject to its

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

storage capacity constraints

$$\sum_{j \in X} S_j \leq S, \tag{1}$$

where S_j denotes the memory size of the inference model. When an end device $ED_i \in N$ opts to offload part or all of its model inference task to the edge server, its offloading decision is represented by $a_i \in \{0, 1\}$, where $a_i = 1$ indicates end-edge collaborative inference and $a_i = 0$ indicates local inference. To optimize resource allocation and pricing, the edge service provider employs backward induction to anticipate ED_i's optimal offloading decision a_i and model partition point d_i, as detailed in Lemma 1 and Proposition 1. Based on these anticipated responses, the service provider dynamically sets the resource allocation $\rho_i = [f_i, w_i]$, comprising computational resources f_i and bandwidth w_i, and the corresponding fee $\pi_i \geq 0$ to maximize its utility while ensuring ED_i's cost efficiency, as defined in (13). This ensures that the allocated resources and fees are tailored to the expected resource consumption determined by a_i and d_i.

B. Local Inference Model

If ED_i decides to perform model inference locally, the task will require the use of the local computation resources of the end device. The time required to perform the inference task locally is defined as

$$r_i^l = \frac{\sum_{j=0}^{L_i} F_i[j]}{f_i^l}, \tag{2}$$

where f_i^l denotes the local computation resources of ED_i. For simplicity, we abstract computing resources into Floating-Point Operations Per Second (FLOPS), supporting diverse hardware [31].

C. Collaborative Inference Model

If ED_i chooses to offload part or all of the model inference tasks to the edge server (i.e., $a_i = 1$), its model inference time consists of three parts: the local inference time on the end device, the data transmission time between the end device and the edge server, and the inference time on the edge server [32]. The local inference time for ED_i is defined as

$$r_i^l(d_i) = \frac{T_i^1(d_i)}{f_i^l}, \tag{3}$$

where T_i^1 represents the number of floating-point operations required for the portion of the model that ED_i needs to infer locally. Similarly, the inference time for ED_i on the edge server is given by

$$r_i^e(d_i) = \frac{T_i^2(d_i)}{f_i}, \tag{4}$$

where T_i^2 represents the number of floating-point operations required for the portion of the model that ED_i needs to infer on the edge server. We assume that end devices and edge servers perform computational tasks with resources measured in FLOPS [32]. A unit of computational resource here is equivalent to 1 FLOPS. Moreover, T_i^1 and T_i^2 are given by

$$T_i^1(d_i) = \sum_{j=0}^{d_i} F_i[j], \tag{5}$$

$$T_i^2(d_i) = \sum_{j=d_i+1}^{L_i} F_i[j], \tag{6}$$

where $d_i \in \mathbb{Z} \cap [0, L_i]$ is the model partition point of the CNN model M_i that ED_i infers when $a_i = 1$, determining the division of computational tasks between the local device and the edge server. Moreover, considering that the computational load of certain layers (such as pooling layers, batch normalization layers, etc.) is negligible, we ignore the delay from those layers [33], [34], [35]. During the collaborative inference process, the end device needs to transmit either raw data or intermediate results to the edge server through a wireless channel, via an access point, after which inference is performed on the edge server. We use w_i to denote the bandwidth allocated to ED_i by the edge service provider, and we assume that the transmission is based on a Gaussian channel. The transmission time for ED_i can be expressed using the Shannon formula [36]

$$r_i^u(d_i, w_i) = \frac{O_i[d_i]}{w_i \log_2\left(1 + \frac{p_i h_i}{\sigma_i^2 + I_c}\right)}, \tag{7}$$

where h_i is the channel coefficient between ED_i and the edge server, p_i is the transmission power of ED_i, I_c is the average co-channel interference power, and σ_i^2 is the noise power at the edge server. This transmission rate model corresponds to Orthogonal Frequency-Division Multiple Access (OFDMA) [37], a technology adopted in 5G and Wi-Fi 6, which allocates resource blocks to EDs for data transmission. By using non-overlapping subcarriers, OFDMA avoids intra-cell interference during simultaneous transmissions by multiple EDs. Similar to prior studies [38], [39], [40], we make a common assumption that the transmission time required for the edge server to return the computed results to the ED can be neglected, as the output data size for CNN model inference tasks is significantly smaller than the raw data or the intermediate transmission data.

IV. Problem Formulation

In this section, we first formulate the interaction process between EDs and the edge service provider as a Stackelberg game. Subsequently, we derive the utility functions and optimization objectives for both the MEC server and the EDs.

A. The Construction of the Stackelberg Game

In this paper, we aim to optimize the utilities of both the edge service provider and the EDs. To ensure the Quality of Experience (QoE) for the EDs, the problem needs to account for both energy consumption and inference latency within the system. EDs can determine whether to offload tasks to the edge service provider, as well as the number of model layers to offload. Therefore, task allocation at the end device side should

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

be conducted to minimize the overall cost. Specifically, these costs include the energy consumption for local inference, the energy consumption for uploading inter-layer data, and the cost paid to the edge service provider for resource usage. On the edge service provider's side, the objective is to maximize revenue by selling bandwidth and computation resources to the EDs.

In this Stackelberg game, the edge service provider, as the leader, optimizes the caching decision X, resource allocation $\rho = \{\rho_i\}$, and pricing strategy $\pi = \{\pi_i\}$ to maximize its utility, as defined in (13). The service provider employs backward induction to compute the optimal responses of the EDs, including their offloading decisions $a = \{a_i\}$ and model partition points $\{d_i\}$, based on Lemma 1. Specifically, for each ED_i, the pricing π_i and resource allocation $\rho_i = [f_i, w_i]$ are dynamically determined by anticipating ED_i's cost-minimizing response, as derived in Proposition 1, ensuring alignment with the actual resource consumption driven by a_i and d_i. The EDs, as followers, observe these strategies and select their offloading decisions $a_i \in \{0, 1\}$ and model partition points d_i to minimize their costs, as defined in (11), subject to the constraints in (11a)–(11c). The asymmetry in this participant scenario, where one party takes the lead while assuming the other party will make rational choices based on its decision, aligns with the Stackelberg game model. Thus, the interaction between the edge service provider and the EDs can be viewed as a single-leader, multi-follower Stackelberg game, where the EDs act as followers and the edge service provider assumes the role of the leader.

B. ED Cost

The inference task incurs both energy consumption and economic expenditure for the ED. In the case of local inference, the cost depends on the energy consumed by the ED while executing the CNN inference task, which is modeled as

$$C_i^0 = r_i^l \left(f_i^l\right)^2 k_i^l \gamma_i \tag{8}$$

where r_i^l is the local inference time, f_i^l represents the local computing capability of ED$_i$, k_i^l is the energy consumption parameter specific to the ED's hardware, and γ_i is the unit local energy cost, converting energy consumption into monetary cost. The term $(f_i^l)^2$ reflects the quadratic relationship between computing frequency and power consumption, as power scales with the square of the frequency in CMOS-based processors [41]. This quadratic model, widely used in edge computing for tasks like CNN inference [1], [13], [42], [43], captures the energy consumption of local inference when combined with r_i^l, scaled by $k_i^l \gamma_i$ to obtain the monetary cost.

In the case of collaborative inference, we define the collaborative inference cost as

$$C_i^1(d_i,\ \rho_i, a_{-i}) = r_i^l\left(d_i\right)\left(f_i^l\right)^2 k_i^l \gamma_i + r_i^u\left(di,\ w_i\right) p_i \beta_i \gamma_i + \pi_i \tag{9}$$

where $r_i^l(d_i)$ is the local inference time for the portion of the CNN model processed by ED_i, defined in (3), $r_i^u(d_i, w_i)$ is the transmission time for sending data to the edge server, defined in (7), β_i is the transmission antenna power efficiency parameter of ED_i, $(f_i^l)^2 k_i^l \gamma_i$ and $p_i \beta_i \gamma_i$ convert the local inference and transmission energy consumption into monetary costs, respectively, and π_i is the fee charged by the edge server, all in monetary units. The offloading decisions $a_{-i} = \{a_j \mid j \in N, j \neq i\}$ affect ED_i's cost indirectly through the edge server's resource allocation strategy. In the Stackelberg game, the edge service provider sets the pricing π_i and resource allocation $\rho_i = [f_i, w_i]$ prior to the EDs' offloading decisions, anticipating the potential resource competition caused by other EDs' offloading choices.

The total cost for ED_i is given by

$$C_i\left(a_i, d_i, \rho_i, a_{-i}\right) = \left(1 - a_i\right) C_i^0 + a_i C_i^1\left(d_i,\ \rho_i, a_{-i}\right) \tag{10}$$

Equation (10) defines the total cost of end device ED_i, where C_i^0 represents the cost of local inference and $C_i^1(d_i, \rho_i, a_{-i})$ denotes the cost of collaborative inference. The EDs' preference is to select the strategy that minimizes their total cost, determined by comparing C_i^0 with $C_i^1(d_i, \rho_i, a_{-i})$. This selection tendency is directly embedded in the optimization objective of problem (11), which seeks to minimize $C_i(a_i, d_i, \rho_i, a_{-i})$ by adjusting the offloading decision a_i and model partition point d_i, subject to time and resource constraints.

C. Problem Formulation

We assume that the ED and the edge service provider are rational entities. The objective of ED_i is to minimize its cost while meeting its completion time requirements, model partitioning strategy, and the service provider's caching decisions. Therefore, ED_i aims to solve

$$\min_{d_i, a_i} \quad C_i\left(a_i, d_i, \rho_i, a_{-i}\right) \tag{11}$$

$$\text{s.t.} \quad a_i\left(r_i^l(d_i) + r_i^e(d_i) + r_i^u(d_i, w_i)\right) \leq r_i^l, \tag{11a}$$

$$a_i = 0, \quad M_i \notin X, \tag{11b}$$

$$d_i \in \mathbb{Z} \cap [0, L_i], \tag{11c}$$

where the constraint (11a) ensures that the task completion time of ED_i during collaborative inference does not exceed the task's deadline. The constraint (11b) ensures that ED can only opt for collaborative inference if its CNN model is cached by the service provider, thus preventing cold starts for the end device's inference tasks. The constraint (11c) ensures that the model's partition point is within the correct range. We define $a_i \in A_i = \{0, 1\}$ as the action set of ED_i, where 0 represents local inference and 1 represents collaborative inference.

Consistent with widely used pricing models, we assume that the service provider's revenue depends on the pre-determined price π_i it sets for collaborative inference and the EDs' subsequent offloading decisions. Therefore, the utility the service provider gains from the EDs engaging in collaborative inference is given by

$$U_X\left(a, \rho, \pi\right) = \sum_{i \in N} a_i \mathbb{I}_{M_i \in X} \pi_i \tag{12}$$

where $\mathbb{I}_{M_i \in X}$ is an indicator function. We define $a = (a_i)_{i \in N}$ as the offloading decisions of the EDs, $\rho = (\rho_i)_{i \in N}$ as the resource allocation decisions, and $\pi = (\pi_i)_{i \in N} \in [0, \bar{\pi}]^N$ as the

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

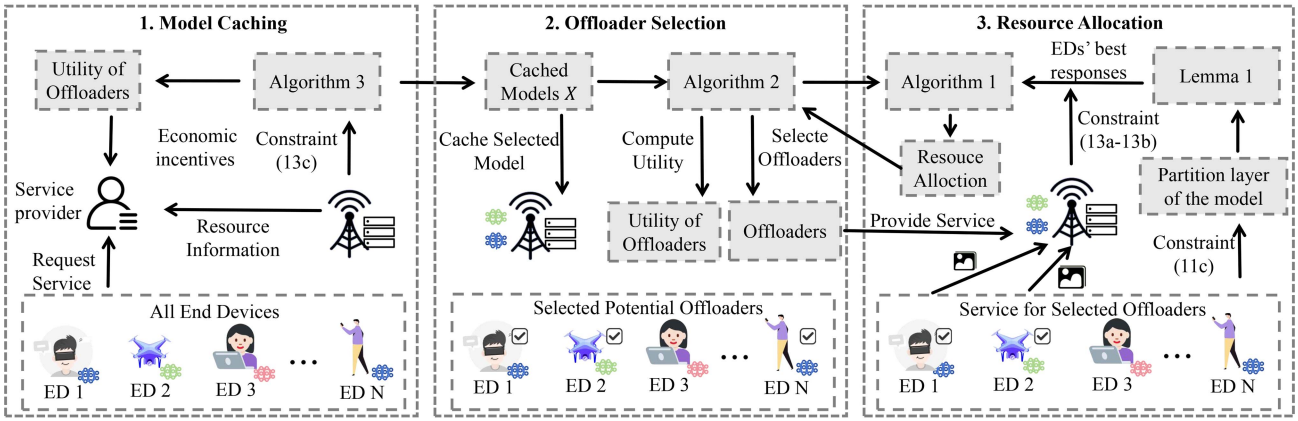


Fig. 2. Main structure of DISCNN.

pricing decisions, where $\bar{\pi} \in \mathbb{R}$ is a sufficiently large constant representing the price cap. We assume that the service provider's objective is to maximize its utility by selecting the resource allocation ρ, pricing π, and caching decisions X. Therefore, the service provider aims to solve

$$\max_{\rho,\pi,X\subseteq J} \quad U_X(a,\rho,\pi) \tag{13}$$

$$\text{s.t.} \quad \sum_{i\in N} f_i \le F, \tag{13a}$$

$$\sum_{i\in N} w_i \le W, \tag{13b}$$

$$\sum_{j\in X} S_j \le S, \tag{13c}$$

This problem is a multi-follower, single-leader Stackelberg game, where the edge service provider acts as the leader and the EDs are the followers. We refer to this problem as the Pricing, Caching, Resource Allocation, and Model Partitioning game (PCRP). Our focus is on the existence of a Stackelberg equilibrium and the complexity of computing the equilibrium under complete information, where the system parameters and utilities are known. Clearly, problems (11) and (13) are intricately coupled: the resource allocation, pricing, and caching strategies of the edge service provider influence the offloading decisions of the EDs, while the offloading decisions of the EDs, in turn, affect the edge service provider's decisions by impacting its revenue.

V. Algorithm Design and Analysis

In this section, we design the algorithmic framework based on backward induction and elaborate on its components and complexity analysis.

A. Overview of the Algorithmic Framework

As illustrated in Fig. 2, the algorithmic framework of DisCNN comprises three main steps, designed based on the interdependencies among the algorithms, adopting a staged decision-making process to reduce computational complexity and enhance scalability in dynamic environments, thereby enabling progressive optimization of decisions under resource constraints. First, the edge service provider determines the model caching decision using Algorithm 3, selecting a subset of models from the set of all EDs based on model utility density to maximize potential utility within storage constraints. Second, given the caching decision and the potential set of offloading devices, the edge service provider employs Algorithm 2 to select an optimal subset of offloading devices, optimizing the overall utility of the service provider by evaluating each device's utility-to-resource ratio, with its optimal approximation ratio proven to ensure selection performance. Finally, for the selected set of offloading devices, the edge service provider invokes Algorithm 1 to allocate computational and communication resources and computes each device's response based on Lemma 1, maximizing utility under resource constraints while delivering distributed inference services.

The details of these steps will be further discussed below.

B. Analysis of EDs' Best Responses and Equilibrium Existence

We begin our analysis by presenting the best response characteristics of the EDs given the service provider's caching decision X, pricing scheme π, and resource allocation strategy ρ. For the caching decision X, we denote the set of EDs with cached models as $N_X = \{i \in N | M_i \in X\}$, representing the potential offloaders. We first demonstrate that the best response of each ED follows a threshold structure, which can be efficiently computed.

Lemma 1: Consider an $\text{ED}_i \in N_X$. Let d_i^* be the model partition point that satisfies the condition $r_i^l(d_i) + r_i^e(d_i) + r_i^u(d_i, w_i) \le r_i^l$. If $\nexists d_i^* \in \mathbb{Z} \cap [0, L_i]$, then $a_i^* = 0$, otherwise

$$a_i^* = \begin{cases} 1, & \pi_i \le T_i, \\ 0, & \text{else}, \end{cases} \tag{14}$$

where $T_i = r_i^l(f_i^l)^2 k_i^l \gamma_i - r_i^l(d_i^*)(f_i^l)^2 k_i^l \gamma_i - r_i^u(d_i^*, w_i) p_i \beta_i \gamma_i$.

Proof: When the service provider has made the relevant decisions, ED_i should determine, based on the provider's decisions,

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

whether collaborative inference can meet the task deadline for model inference. Specifically, ED_i seeks to find a d_i^* such that $r_i^l(d_i) + r_i^e(d_i) + r_i^u(d_i, w_i) \le r_i^l$. If no such partition point exists, it implies that collaborative inference will not yield greater utility. However, if a d_i^* exists and satisfies $C_i^1(d_i, \rho_i, a_{-i}) \le C_i^0$, meaning that the cost of collaborative inference is less than or equal to the cost of local inference, then ED_i will choose collaborative inference. This leads to the following equation

$$a_i^* = \begin{cases} 1, & r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l \gamma_i \quad + r_i^u\left(d_i^*, w_i\right) p_i \beta_i \gamma_i \\ & \qquad + \pi_i \le r_i^l \left(f_i^l\right)^2 k_i^l \gamma_i, \\ 0, & \text{else}, \end{cases} \tag{15}$$

Thus, we obtain $\pi_i \le r_i^l(f_i^l)^2 k_i^l \gamma_i - r_i^l(d_i^*)(f_i^l)^2 k_i^l \gamma_i - r_i^u(d_i^*, w_i) p_i \beta_i \gamma_i$, which proves the result. □

The edge service provider predicts the set of offloading decisions $a = \{a_i\}$ and the corresponding model partition points d_i^* for all EDs by computing their optimal responses ((14)), based on the complete information assumption, where the edge service provider has access to the edge devices' computational capabilities, energy consumption parameters, and task deadlines, requiring this information to ensure efficient collaborative inference and to formulate effective decision-making strategies through device information analysis.

From Lemma 1, it can be concluded that when the service provider has determined the caching decision X, pricing π, and resource allocation strategy ρ, the best response of the ED can be computed. Considering the best response of the ED, we proceed to prove the existence of Subgame Perfect Equilibrium (SPE), which is defined as follows

Definition 1: Assume that (ρ^*, π^*, X^*) is a solution to problems (13), and let a^* be a solution to problems (11). If for all feasible solutions (ρ, π, X, a), the following condition holds, then the solution $(\rho^*, \pi^*, X^*, a^*)$ is the Subgame Perfect Equilibrium (SPE) for the PCRP.

$$\begin{aligned} &U_{X^*}(a^*, \rho^*, \pi^*) \ge U_X(a^*, \rho, \pi), \\ &C_i(a_i^*, d_i^*, \rho_i^*, a_{-i}^*) \le C_i(a_i, d_i, \rho_i^*, a_{-i}^*), \\ &\forall \{a_i, d_i\} \in A_i \times (\mathbb{Z} \cap [0, L_i]), \quad \forall i \in N. \end{aligned} \tag{16}$$

Theorem 1: There exists a SPE in the PCRP game.

Proof: By Lemma 1, for a given (ρ, π, X), the optimal response a^* of the EDs is unique and can be efficiently computed. Therefore, according to the extreme value theorem [44], a solution to problems (13) exists, and by Definition 1, this solution is a SPE, which concludes the proof. □

It is observed that the service provider can use Theorem 1 to compute the SPE. The existence of a SPE ensures that the edge service provider can predict the rational responses of EDs, including their offloading decisions a_i^* and model partition points d_i^*, through backward induction, and accordingly formulate optimal strategies to maximize its utility while satisfying the cost minimization objectives of the edge devices. We now proceed to analyze the complexity of computing the SPE.

Algorithm 1: Server Resource Allocation (SRA).

Input: The offloading devices N_X^o, total bandwidth W, total computation F

Output: The utility $U_X^{N_X^o}$, bandwidth allocations W_i_list, computation allocations F_i_list

Initialize $U_X^{N_X^o} \leftarrow 0$, $W_i\_list \leftarrow \emptyset$, $F_i\_list \leftarrow \emptyset$
Prepare N_X^o as GPU tensors
for $i \leftarrow 1$ to $|N_X^o|$ **do**
  $w_i\_final \leftarrow 0$, $f_i\_final \leftarrow 0$, $\pi_i\_max \leftarrow -\infty$
  Generate tensor grid $\{w_i, f_i\}$ from ratios $\{g, 2g, \ldots, 1\} \cdot \{W, F\}$
  Compute $\{d_i\}$ for grid $\{w_i, f_i\}$ via `user_response` (batched)
  Filter grid $\{w_i, f_i, d_i\}$ where delay $\le$ deadline (batched)
  Compute $\pi_i \leftarrow C_i^0 - r_i^l(d_i)(f_i^l)^2 k_i^l \gamma_i - r_i^u(d_i, w_i) p_i \beta_i \gamma_i$ (batched)
  Select $\{w_i\_best, f_i\_best\}$ with max π_i
  $W_i\_list \leftarrow W_i\_list \cup \{w_i\_best\}$, $F_i\_list \leftarrow F_i\_list \cup \{f_i\_best\}$
  $W \leftarrow W - w_i\_best$, $F \leftarrow F - f_i\_best$
  $U_X^{N_X^o} \leftarrow U_X^{N_X^o} + \pi_i\_max$
end
return $U_X^{N_X^o}, W_i\_list, F_i\_list$

C. Resource Allocation and Pricing Optimization for Fixed Offloading Devices

We first consider a feasible caching decision X by the service provider, i.e., $\sum_{j \in X} S_j \le S$, along with a set of offloading devices $N_X^o = \{i \in N_X | a_i = 1\}$. Our focus is on determining the resource allocation and pricing decisions that, given the caching decision X and the offloading set N_X^o, maximize the service provider's utility $U_X^{N_X^o}$.

For a given caching decision X and a set of offloading devices N_X^o, the edge service provider's optimal utility $U_X^{N_X^o}$ is the solution to the following problem

$$\max_{(\pi_i, \rho_i)_{i \in N_X^o}} \quad \sum_{i \in N_X^o} \pi_i \tag{17}$$

$$\text{s.t.} \quad \sum_{i \in N_X^o} f_i \le F, \quad \sum_{i \in N_X^o} w_i \le W, \tag{17a}$$

$$f_i \ge f_i^l, \quad w_i \ge 0, \quad \forall i \in N_X^o, \tag{17b}$$

$$\exists d_i^* \in \mathbb{Z} \cap [0, L_i], \quad \forall i \in N_X^o, \tag{17c}$$

$$r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l \gamma_i + r_i^u\left(d_i^*, w_i\right) p_i \beta_i \gamma_i + \pi_i \le C_i^0, \quad \forall i \in N_X^o, \tag{17d}$$

Constraint (17a) ensures the validity of the decisions made for this fixed set of offloading devices, while Constraints (17b)–(17d) are the necessary conditions to guarantee that the EDs can offload.

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

We first present the characteristics of the service provider's optimal resource pricing, denoted as $\pi^* = \{\pi_i{}^*\}_{i\in N}$.

Proposition 1: When problem (17) has a solution for the given set of offloading devices, there exists a set of resource allocation decisions ρ and resource pricing π such that the EDs $i \in N_X^o$ can offload. For $i \in N_X^o$, the edge service provider's optimal pricing strategy for the user is $\pi_i{}^* = C_i^0 - r_i^l(d_i^*)(f_i^l)^2 k_i^l \gamma_i - r_i^u(d_i^*, w_i^*) p_i \beta_i \gamma_i$, where the optimal resource allocation strategy $\{(f_i^*, w_i^*)\}_{i\in N_X^o} = \rho_i^*$ is the solution to the following problem

$$\min_{(\pi_i,\rho_i)_{i\in N_X^o}} \quad \sum_{i\in N_X^o} r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l\gamma_i + r_i^u\left(d_i^*, w_i\right) p_i\beta_i\gamma_i, \tag{18}$$

$$\text{s.t.} \quad \sum_{i\in N_X^o} f_i \le F, \quad \sum_{i\in N_X^o} w_i \le W, \tag{18a}$$

$$f_i \ge f_i^l, \quad w_i \ge 0, \quad \forall i \in N_X^o, \tag{18b}$$

$$\exists d_i^* \in \mathbb{Z} \cap [0, L_i], \quad \forall i \in N_X^o, \tag{18c}$$

$$r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l\gamma_i + r_i^u\left(d_i^*, w_i\right) p_i\beta_i\gamma_i \le C_i^0, \quad \forall i \in N_X^o, \tag{18d}$$

Proof: Observing the optimal solution to problems (17), constraint (17d) is always active. Therefore, the optimal pricing satisfies $\pi_i{}^* = C_i^0 - r_i^l(d_i^*)(f_i^l)^2 k_i^l\gamma_i - r_i^u(d_i^*, w_i^*)\beta_i\gamma_i$. Subsequently, since $\sum_{i\in N_X^o} C_i^0$ is constant, the original problem (17) is equivalent to the problem (18), which concludes the proof. □

The significance of Proposition 1 lies in the fact that calculating the edge service provider's optimal pricing strategy requires obtaining the optimal resource allocation strategy $\{(f_i^*, w_i^*)\}_{i\in N_X^o}$ for the fixed set of offloading devices, in order to minimize the total local inference cost and transmission cost. Moreover, from Proposition 1, the optimal pricing π_i^* is directly related to the model partition d_i^* and allocated resources (f_i^*, w_i^*). As d_i^* increases, more computation is executed locally by the end device, reducing the edge-side workload and transmission volume, thus leading to a lower π_i^*. This indicates that the proposed pricing mechanism is inherently usage-aware, reflecting the actual resource consumption rather than a fixed fee.

This problem is essentially a mixed-integer nonlinear programming (MINLP) problem, making it NP-hard and challenging to obtain a feasible closed-form solution. Common solutions leverage deep learning and reinforcement learning to accelerate equilibrium convergence; however, these methods entail high training costs and are thus unsuitable when the number of EDs is large or the system scale is dynamic. Based on the unique optimal response that each ED has to the resource allocation and pricing strategy provided by the edge service provider, we design an efficient algorithm for resource allocation (SRA). The edge service provider's optimal pricing for the EDs is determined according to Proposition 1 and the resource allocation strategy.

As illustrated in Algorithm 1, the edge service provider allocates computational and communication resources to EDs within a fixed set of offloaders, denoted as N_X^o, by generating a tensor grid of allocation ratios with a specified granularity (line 5). Here, granularity g (e.g., $g = 0.1$) refers to the step size used to discretize the allocation ratios, which represent the fractions of total bandwidth W and computational resources F assigned to each ED. Leveraging GPU acceleration, the algorithm computes the optimal model partition points $\{d_i^*\}$ for all allocation combinations in a batched manner, based on the allocated resources and Lemma 1 (line 6). Valid allocations satisfying delay constraints are filtered in parallel (line 7), and the corresponding utilities are computed batched (line 8). The allocation yielding the maximum utility is selected for each ED (line 9). After evaluating all feasible allocation ratios, the edge service provider updates its remaining computational and communication resources (lines 10 to 12) before proceeding to the next offloader in the set.

Algorithm 2: Singleton Utility Maximization (SUM).

Input: The cached model X, the potential offloaders N_X

Output: The offloaders N_X^o, the utility generated by offloaders U_X^{SUM}

1 Initialize $N_X^o \leftarrow \emptyset$, $U_X^{\text{SUM}} \leftarrow 0$
2 Prepare N_X as GPU tensor indices
3 **for** $i \leftarrow 1$ **to** $|N_X|$ **do**
4 $\quad U_X^i, w_i, f_i \leftarrow \text{SRA}(i, W, F)$ (batched)
5 $\quad i^* \leftarrow \arg\max_{i\in N_X} \frac{U_X^i}{w_i + f_i}$
6 $\quad N_X^o \leftarrow \arg\max_{N_X^o \in \{N_X^o, N_X^o \cup \{i^*\}\}} U_X^{N_X^o}$
7 $\quad N_X \leftarrow N_X \setminus \{i^*\}$
8 **end**
9 $U_X^{SUM} \leftarrow U_X^{N_X^o}$
10 **return** U_X^{SUM}, N_X^o

D. Determining the Optimal Set of Offloading Devices

In this subsection, we focus on how to select a set of offloading devices that maximizes the edge service provider's utility, given a caching decision. Specifically, we aim to solve the following problem

$$\max_{N_X^o \subseteq N_X} \quad \max_{(\rho_i)_{i\in N_X^o}} \sum_{i\in N_X^o} \Big(C_i^0 - r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l\gamma_i - r_i^u\left(d_i^*, w_i\right) p_i\beta_i\gamma_i\Big), \tag{19}$$

$$\text{s.t.} \quad \sum_{i\in N_X^o} f_i \le F, \quad \sum_{i\in N_X^o} w_i \le W, \tag{19a}$$

$$f_i \ge f_i^l, \quad w_i \ge 0, \quad \forall i \in N_X^o, \tag{19b}$$

$$\exists d_i^* \in \mathbb{Z} \cap [0, L_i], \quad \forall i \in N_X^o, \tag{19c}$$

$$r_i^l\left(d_i^*\right)\left(f_i^l\right)^2 k_i^l\gamma_i + r_i^u\left(d_i^*, w_i\right) p_i\beta_i\gamma_i \le C_i^0, \quad \forall i \in N_X^o, \tag{19d}$$

For a given N_X^o, the inner maximization problem can be computed as shown in the previous section. Therefore, problem (37)–(41) is essentially a set function maximization problem

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

over the ground set N_X, which can be equivalently written as

$$U_X^{N_X^o{}^*} = \max_{N_X^o \subseteq N_X} U_X^{N_X^o} \tag{20}$$

Problem (20) is NP-hard. Before introducing our approach to solving this problem, we first present two properties of set functions.

Definition 2: Monotonicity: Let $f : 2^E \to \mathbb{R}$ be a set function defined on the power set of a finite set E. If for any subsets $A \subseteq B \subseteq E$, we always have $f(A) \leq f(B)$, then the set function f is called monotone non-decreasing, meaning the function value does not decrease as elements are added to the set.

Definition 3: Submodularity: Let $f : 2^E \to \mathbb{R}$ be a set function defined on the power set of a finite set E. If for any subsets $A \subseteq B \subseteq E$ and element $e \in E \setminus B$, the following holds

$$f(A \cup e) - f(A) \geq f(B \cup e) - f(B) \tag{21}$$

The set function f is then called submodular, meaning that as elements are added to the set, the marginal gain from adding the same element does not increase.

When the given function is a monotone submodular function, existing approximation algorithms can effectively guarantee an approximation ratio bound of 1/2, such as the marginal benefit maximization algorithm based on greediness. However, in the current scenario, the edge service provider's utility function U is neither monotone nor submodular. As the number of EDs increases, the utility function may decrease, and due to the differences in EDs' performance and varying task requirements, submodularity is difficult to ensure for this function. Given these limitations, we propose an efficient greedy-based approximation algorithm and prove that it has a bounded approximation ratio, providing worst-case accuracy guarantees in non-monotone, non-submodular settings.

As described in Algorithm 2, the edge service provider applies SRA to the potential offloading set, leveraging GPU tensor indices, to determine the utility and resource allocation strategy for each user in a batched manner (line 4). SUM then identifies the potential offloader with the highest utility-to-total-resource ratio (line 5). Next, the algorithm evaluates the change in utility resulting from offloading this user by comparing candidate offloading sets (line 6). If the utility increases, the user is added to the offloading set (line 6). Regardless of whether the user is added, they are removed from the potential offloading set (line 7). This process repeats until the potential offloading set is empty.

In the following part, we will prove the approximation ratio bound of the proposed SUM algorithm.

Theorem 2: Let N_X^* be the optimal offloading set under the caching decision X of the edge service provider, and ρ^* be the optimal resource allocation strategy for these EDs. Then, $U_X^{N_X^*}$ represents the optimal utility of the edge service provider, which is clearly the sum of the fees paid by the users in N_X^*. We denote the utility that user i in N_X^* brings to the service provider as $U(N_X^*(i))$, which is equal to π_i. Let the total computation and communication resources allocated by the service provider to user i in N_X^* be denoted as $R(N_X^*(i))$. Let N_X^{SUM} be the offloading set obtained by applying the SUM algorithm under the caching decision X. The SUM algorithm has an approximation ratio bound of $\frac{R(N_X^{SUM})}{N_X^* R(N_X^*)}$.

Algorithm 3: Model Utility Maximization Selection (MUMS).

Input: EDs N, user model J, total bandwidth W, total computation F, server memory limit S
Output: The cached model X, the offloaders N_X^o, the utility $U_X^{N_X^o}$

1 Initialize $X \leftarrow \emptyset$, $U_X^{N_X^o} \leftarrow 0$
2 Prepare J as GPU tensor indices
3 **while** $\sum_{j \in X} s_j \leq S \wedge J \neq \emptyset$ **do**
4 $\quad j^* \leftarrow \arg\max_{j \in J} \frac{U_j^{\text{SUM}}}{s_j}$
5 $\quad$ **if** $S \geq \sum_{j \in X \cup \{j^*\}} s_j$ **then**
6 $\quad\quad X \leftarrow X \cup \{j^*\}$
7 $\quad$ **end**
8 $\quad J \leftarrow J \setminus \{j^*\}$
9 **end**
10 $N_X \leftarrow \{n \in N \mid n \text{ uses model } j \in X\}$
11 $N_X^o, U_X^{N_X^o} \leftarrow \texttt{SUM}(X, N_X)$
12 **return** $X, N_X^o, U_X^{N_X^o}$

Proof: Based on the definition of utility, we can derive the following proof process

$$\frac{U_X^{N_X^*}}{R(N_X^*)} = \sum_{i \in N_X^*} \frac{U(N_X^*(i))}{R(N_X^*(i))} \leq N_X^* \frac{U(N_X^*(i^*))}{R(N_X^*(i^*))} \leq N_X^* \frac{U(j^*)}{R(j^*)}$$

$$\Longrightarrow \frac{U_X^{N_X^*}}{N_X^* R(N_X^*)} \leq \frac{U(j^*)}{R(j^*)} \leq \frac{U_X^{N_X^{SUM}}}{R\left(N_X^{N_X^{SUM}}\right)}$$

$$\Longrightarrow \frac{R\left(N_X^{SUM}\right) U_X^{N_X^*}}{N_X^* R(N_X^*)} \leq U_X^{N_X^{SUM}} \tag{22}$$

Where $i^* = \arg\max_{i \in N_X^*} \frac{U(N_X^*(i))}{R(N_X^*(i))}$ and $j^* = \arg\max_{j \in N_X} \frac{U(j)}{R(j)}$. The key point is that lines 4 to 5 in the algorithm ensure that the optimal offloading set obtained in the first iteration of the SUM algorithm must include the user with the highest utility-to-resource ratio among all potential offloading users. This guarantees the bounded approximation ratio of the algorithm, thus concluding the proof. □

E. Optimal Caching Strategy

Finally, we focus on determining the caching decision X that maximizes the service provider's utility, as shown below

$$X^* = \arg\max_{X \subseteq J} U_X^{N_X^o{}^*} \tag{23}$$

$$\text{s.t.} \quad \sum_{j \in X} S_j \leq S, \tag{23a}$$

The caching decision made by the edge service provider is NP-hard. Although in the previous section we showed that the utility function of the edge service provider with respect to the selection of the offloading set is neither monotonic nor submodular, in this

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

section, we will prove that the utility function is monotonic with respect to the number of cached models.

Proposition 2: Let $A \subseteq J$, where A represents the current set of cached models, and $j \in J/A$, then

$$U_A^{N_A^{o*}} \leq U_{A\cup j}^{N_{A\cup j}^{o*}} \tag{24}$$

Proof: We will prove this by contradiction. Suppose $U_A^{N_A^{o*}} > U_{A\cup\{j\}}^{N_{A\cup\{j\}}^{o*}}$. Clearly, $N_A^{o*} \subseteq A \subseteq A \cup \{j\}$. If $U_A^{N_A^{o*}} > U_{A\cup\{j\}}^{N_{A\cup\{j\}}^{o*}}$, it implies that the set N_A^{o*} would provide greater utility when the edge service provider adopts the caching decision $A \cup \{j\}$. However, if this is the case, then the set $N_{A\cup\{j\}}^{o*}$ cannot be the optimal offloading set under the caching decision $A \cup \{j\}$, which contradicts the assumption that $N_{A\cup\{j\}}^{o*}$ is the optimal offloading set under this decision. Therefore, we have $U_A^{N_A^{o*}} \leq U_{A\cup\{j\}}^{N_{A\cup\{j\}}^{o*}}$, i.e., Proposition 2 holds. □

Moreover, since the utility of the service provider exhibits submodularity when determining the offloading set under a given caching decision, it is also difficult to guarantee submodularity when making caching decisions. For the problem of maximizing a monotonic but non-submodular set function, we propose a greedy algorithm based on the ratio of model utility to storage size. As described in Algorithm 3, the algorithm first leverages GPU tensor indices to prepare the model set and uses `SUM` to calculate the total utility each model can provide when cached by the edge service provider (line 4). It then selects the model with the highest ratio of total utility to memory usage (line 4). Provided the server's memory constraints are met, this model is cached (lines 5 to 6). The process repeats until the memory limit is reached or no models remain. Finally, the algorithm determines the offloading users and their utility by invoking `SUM` on the cached models' users (line 11).

F. Complexity Analysis

The computational complexity of DisCNN is primarily determined by the number of models $|J|$, the number of potential offloading devices $|N_X|$, and the resource allocation granularity g, with its core components comprising model caching, offloader selection, and resource allocation. In the model caching step, the process iterates over $|J|$ models to compute each model's user group utility, taking $O((1/g)^2)$ time per computation for a total of $O(|J| \cdot (1/g)^2)$, followed by sorting the models in descending order by their utility-to-size ratio in $O(|J| \log |J|)$ time, and then greedily selecting models until the storage space S is exhausted in $O(|J|)$ time, resulting in a total complexity of $O(|J| \cdot (1/g)^2 + |J| \log |J|)$. The offloader selection step evaluates the utility of each of the $|N_X|$ potential offloading devices iteratively, with each evaluation taking $O((1/g)^2)$ time, leading to a total complexity of $O(|N_X| \cdot (1/g)^2)$. The resource allocation step assigns resources to each of the $|N_X^o|$ selected offloading devices, exploring $(1/g)^2$ resource combinations per device, each requiring $O(L)$ time to compute the user response, where L is the maximum number of model layers treated as a constant, yielding a total complexity of $O(|N_X^o| \cdot (1/g)^2)$, which becomes $O(|N_X| \cdot (1/g)^2)$ in the worst case when $|N_X^o|$ equals $|N_X|$. Consequently, the overall time complexity of the framework is $O(|J| \cdot (1/g)^2 + |J| \log |J| + |N_X| \cdot (1/g)^2)$.

TABLE II
OVERVIEW OF THE SIMULATION PARAMETERS

Symbol	Description	Default value
f_i^l	Local resources of ED_i	Unif[10, 15] GFLOPS
k_i^l	Energy consumption parameter	Unif[0.1, 1] J/Hz/GF^2
γ_i	The unit local energy cost	0.1 \$/J
I_c	Average co-channel interference power	10^{-13} W
h_i	Channel coefficient of ED_i	Unif[0.1, 1]
σ_i^2	Noise at the edge server	Unif[0.5, 1]
β_i	Transmission power efficiency parameter	Unif[0.5, 1]
t_i	The task deadlines of ED_i	Unif[3, 6] s
p_i	Transmission power of ED_i	Unif[10, 100] mW

VI. EXPERIMENTAL RESULTS

In this section, the experimental setup is first described. Following that, we evaluate the performance of the proposed algorithm through extensive simulations.

A. Experiment Settings

Device Configuration: The simulations were conducted on a laptop equipped with an AMD Ryzen 7 8845H with Radeon 780 M Graphics and an NVIDIA GeForce RTX 4070 Laptop GPU, using PyCharm as the development environment.

CNN Model: To evaluate the impact of different number of models on the proposed algorithm, we selected ten widely used convolutional neural network (CNN) models with varying layers and architectures, including AlexNet [45], VGG16, VGG19 [46], ResNet18, ResNet34, ResNet50 [47], GoogleNet [48], DenseNet [49], InceptionV3 [50], and MobileNet [51]. These models were simulated to compute per-layer FLOPS (F_i) and output data sizes (O_i), which determine the computational load and data transmission overhead in the system. These attributes depend solely on model architecture and input resolution, not on dataset content. Therefore, we used the MNIST dataset [52] solely for rapid validation of these attributes.

Parameters of Simulation Environment: To evaluate the performance of the proposed algorithm, we considered a system with up to fifty EDs and up to ten different models. The edge server's bandwidth was set to 20MHz, computational resources were set to 800 GFLOPS, and the caching space for models was set to 1GB. The inference model for each ED was randomly selected from the ten CNN models. In addition, Table II summarizes the initial values of other parameters. These choices of parameters are similar to those used in works [32], [43].

Baselines: In the current scenario, resource allocation, device selection, and model caching are the key components optimized by our proposed algorithm, and they also represent the core factors that determine system performance. To thoroughly evaluate the effectiveness of the proposed algorithm, we have designed baseline comparisons for each of these three aspects:

1) *Resource Allocation Algorithm:* To evaluate the effectiveness of our designed resource allocation algorithm, we considered two widely used methods from previous work:

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

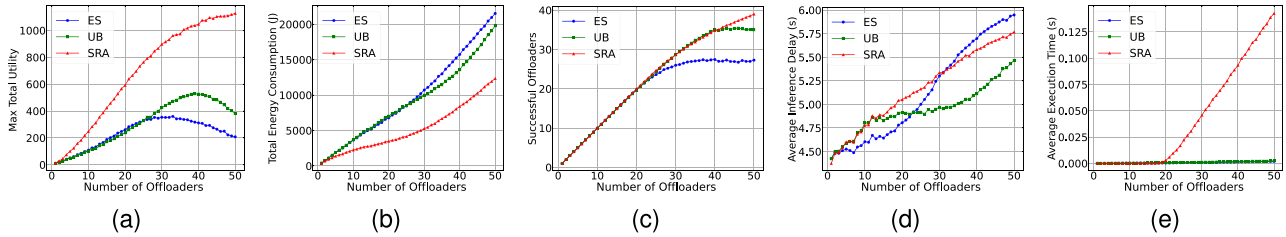


Fig. 3. Performance of resource allocation strategy under varying numbers of offloaders. (a) Total utility of the edge service provider. (b) Total energy consumption of Offloaders. (c) Number of successful Offloaders. (d) Average inference delay of Offloaders. (e) Algorithm execution time.

- *Equal Sharing (ES):* This method evenly distributes both types of resources to the selected offloading devices [53].
- *Urgency-Based (UB):* This method allocates resources based on the urgency of the users' tasks [54], meaning that the shorter the task deadline, the more resources are allocated to that user within the selected offloading set.

2) *Offloading Device Set Selection Algorithm:* To assess the effectiveness of the algorithm for selecting the optimal offloading device set, we also considered two baselines:
   - *Random Greedy Choice (RGC):* This algorithm randomly selects a set of users from the EDs with cached models [55]. If resource allocation to this set of users results in positive utility for the edge service provider, the random selection is deemed successful. If not, another set of users is attempted.
   - *Marginal Utility Maximization (MUM):* This method calculates the marginal utility of each potential offloading ED, selects the ED with the highest marginal benefit, and adds it to the final resource allocation set [56].
3) *Model Caching Algorithm:* Finally, to evaluate the effectiveness and necessity of the model caching algorithm, we considered two baselines. It is important to note that the comparison of model caching algorithms essentially compares the overall framework, as the caching strategy directly influences the performance of the entire offloading and resource allocation process. The algorithms we considered are:
   - *Practical Model Caching (PMC):* This method selects the set of models with the largest number of EDs, while satisfying the storage space constraints of the edge service provider.
   - *Random Model Caching (RMC):* This method randomly selects a set of models that satisfies the storage space constraints of the edge service provider [57].
   - *Real-time Adaptive Partition with Joint Optimization (RAPJM):* This method extends the collaborative inference framework in [58], dynamically optimizing model partition points and resource allocation for mobile environments. The core mechanism employs a weighted joint optimization of quantization loss minimization, inference latency reduction, and transmission latency optimization through its Optimal Split Point Searching algorithm. However, as the original framework lacks consideration for edge server model caching constraints, we integrated it with our MUMS strategy to adapt to storage-limited scenarios.

B. Validation of the Resource Allocation

In Fig. 3(a) to (e), to evaluate the effectiveness of the proposed resource allocation strategy for scenarios involving multiple offloading devices, we assume that the inference models required by different offloading devices have already been cached on the edge server. Moreover, the offloading devices are selected based on Algorithm 2.

Fig. 3(a) shows the utility of the edge service provider with varying numbers of offloaders. As the number of offloaders increases from 1 to 50, the utility obtained by the edge service provider using the SRA algorithm for the offloading set consistently remains at the highest level. Additionally, when the number of offloaders is large, SRA demonstrates greater stability. In contrast, as the number of offloaders increases, the utility of the two baseline algorithms shows a downward trend. This is due to severe resource shortages caused by the increased number of offloaders, making it challenging for the edge service provider to effectively leverage system resources to improve utility.

In Fig. 3(b), we define the energy consumption of EDs as $\sum_{i\in N_X^o} \frac{(1-a_i)C_i^0+a_iC_i^1(d_i,\rho_i,a_{-i})}{\gamma_i}$. The experimental results show that as the number of offloaders increases, the total energy consumption of EDs in the system gradually rises. However, compared to the two baseline algorithms, the SRA algorithm consistently maintains a lower level of energy consumption. By using SRA, the overall energy consumption can be better controlled from a system perspective, even with varying sizes of offloading sets.

Fig. 3(c) shows the number of users able to complete their model inference tasks before the deadline under different resource allocation strategies with varying numbers of offloaders. When the number of offloaders is below 25, the three methods achieve nearly the same number of successful inferences, with all offloaders able to complete their tasks. This is because the edge service provider has sufficient resources, making any strategy sufficient for the current set of offloaders to complete their tasks. However, when the number of users exceeds 25, the SRA and UB methods, which consider offloader task deadlines during resource allocation, enable more offloaders to complete their inference tasks compared to the equal distribution strategy. As the number of offloaders approaches 50, the number of successful

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

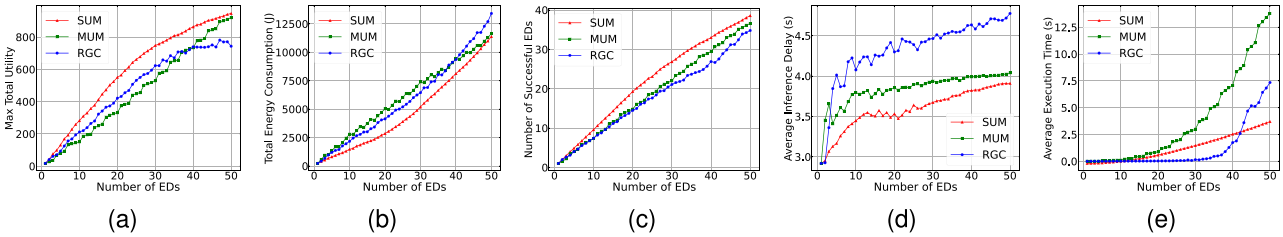


Fig. 4. Performance of offloader selection strategy under varying numbers of EDs. (a) Total utility of the edge service provider. (b) Total energy consumption of EDs. (c) Number of successful EDs. (d) Average inference delay of EDs. (e) Algorithm execution time.

inferences with SRA remains higher than with the two baseline methods, indicating that as the number of offloaders increases, our method continues to ensure the QoE for offloading users.

Fig. 3(d) illustrates the average inference latency for EDs under different resource allocation strategies applied to the offloading device set. As the number of offloading devices increases, the average inference latency for all methods also rises. This is attributed to resource contention in constrained environments, where multiple users compete for limited resources, reducing the amount of resources allocated to individual users and consequently increasing the time required to complete their inference tasks. Although the primary objective of SRA is to ensure the completion of inference tasks for multiple offloading users in resource-constrained scenarios, rather than optimizing average inference latency, the results demonstrate that SRA still provides comparable guarantees of average inference latency to baseline methods. Fig. 3(e) presents the average execution time of different algorithms. As SRA considers the optimal response of various offloading devices to allocate resources, it inevitably increases the resource allocation time to some extent. However, compared to baseline algorithms, the proposed algorithm still achieves resource allocation for individual offloading devices within milliseconds, ensuring its capability for real-time decision-making.

C. Validation of Selecting Set of Offloaders

In Fig. 4(a) to (e), we assume that all EDs use the same model for inference to observe performance of different offloader selection strategy under varying numbers of EDs.

Fig. 4(a) shows that the SUM algorithm consistently achieves higher total utility across varying numbers of EDs. In particular, when the number of EDs is below 40, the utility achieved by the SUM algorithm for the service provider is significantly higher than that of the two baseline algorithms. Furthermore, we observe that while the RGC algorithm incurs minimal computational costs, it struggles to achieve satisfactory utility, particularly when the number of EDs exceeds 40. This demonstrates that our algorithm achieves a strong balance between execution time and utility obtained.

Fig. 4(b) illustrates the overall system energy consumption of three different schemes. Our proposed SUM algorithm significantly reduces total energy consumption across varying numbers of EDs. This is due to the fact that the energy consumption of EDs is closely related to the volume of tasks performed locally, and SUM optimizes the resource allocation by maximizing the benefit for edge service providers. Specifically, it allocates more computational and communication resources to EDs with high computational demands, thereby reducing the local energy consumption of EDs in the system. As mentioned, compared to baseline algorithms, SUM effectively reduces the overall system energy consumption, resulting in energy cost savings for end users.

Fig. 4(c) and (d) illustrate the number of EDs that successfully complete inference tasks and the average inference latency under different offloader set selection strategies. Compared to the two baseline methods, SUM ensures the highest number of EDs completing inference tasks, and this advantage becomes more pronounced as the number of EDs increases. This superiority is attributed to SUM's consideration of both utility and resource allocation ratios, which accounts for the varying resource demands of different EDs. By balancing the service provider's utility with user requirements, SUM effectively guarantees both the quantity of completed user tasks and the quality of task execution.

As shown in Fig. 4(e), when the number of EDs is below 20, the time required by SUM and MUM to determine the offloading set is nearly identical. When the number of EDs exceeds 20, the execution time of the MUM algorithm increases significantly, whereas our proposed algorithm exhibits a much smaller increase. Notably, although the RGC algorithm only needs to identify a feasible offloading set, making it the least computationally intensive solution, its inefficient selection strategy hinders its ability to efficiently determine a suitable offloading set when the number of users is large.

D. Overall Performance of the Proposed DISCNN

In this section, we evaluate the system performance of the proposed framework under varying user scales and multiple models by fixing the number of EDs to 50 and the number of models to 10. Note that the assessment of the overall framework performance inherently includes the evaluation of the proposed caching model algorithm.

In Fig. 5(a), we compare the utility brought to the edge service provider by different algorithms under varying numbers of EDs. We set the number of models to 10. Note that DisCNN employs our proposed caching strategy MUMS and uses SUM for offloader selection, while the RAPJM algorithm utilizes its integrated resource allocation strategy (RAP+JM-MPRA).

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

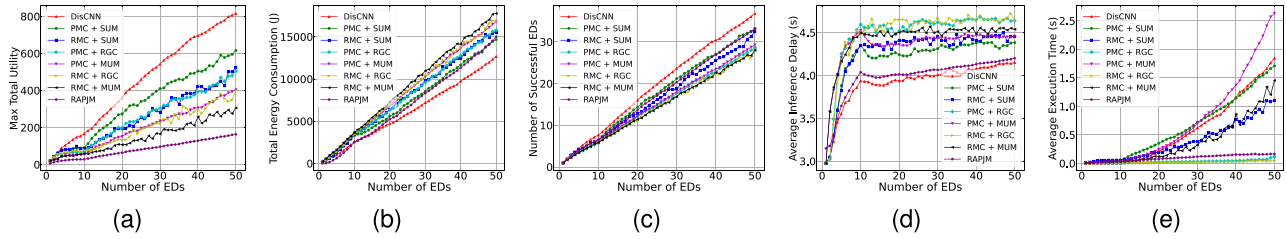


Fig. 5. System performance with respect to varying numbers of EDs, $|J| = 10$. (a) Total utility of the edge service provider. (b) Total energy consumption of EDs. (c) Number of successful EDs. (d) Average inference delay of EDs. (e) Algorithm execution time.

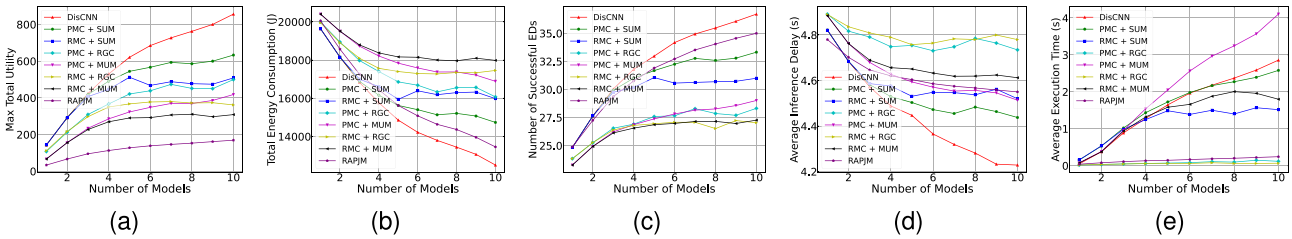


Fig. 6. System performance with respect to varying numbers of Models, $|N| = 50$. (a) Total utility of the edge service provider. (b) Total energy consumption of EDs. (c) Number of successful EDs. (d) Average inference delay of EDs. (e) Algorithm execution time.

All other baseline algorithms apply the SRA resource allocation strategy. The figure shows that the DisCNN framework, through joint optimization of caching, resource allocation, and pricing, achieves significantly higher utility for the edge service provider compared to baseline algorithms. Even when PMC and RMC adopt the SUM offloader selection algorithm, DisCNN outperforms both. This demonstrates the clear advantages of our joint optimization algorithm over popularity-based caching and random caching strategies.

Fig. 6(a) shows the utility gained by the edge service provider under different offloading strategies with varying numbers of models, with the number of EDs set to 50 to better compare the impact of different model counts on the proposed algorithm under multi-user conditions. We observe that DisCNN consistently achieves the highest utility across different model counts, particularly when the model count exceeds 3, where the utility is significantly higher than that of other algorithms. This demonstrates that the performance of DisCNN improves as the variety of models used for ED inference tasks increases. In contrast, RAPJM demonstrates relatively low utility for the edge service provider, as its optimization framework primarily focuses on minimizing end-to-end latency from the end-user perspective, without adequately incorporating service provider revenue considerations into its core objectives. Notably, when baseline caching algorithms adopt our recommended offloader selection algorithm SUM, the utility achieved is significantly higher than with other offloader selection methods, further validating the effectiveness of our proposed offloader selection algorithm.

In Figs. 5(b) and 6(b), we compare the total energy consumption of the system under different numbers of EDs and different numbers of models, setting the number of models to 10 and the number of EDs to 50, respectively. As shown in the figures, DisCNN achieves the lowest energy consumption in both scenarios, with this advantage becoming more pronounced as the number of users and models increases. This indicates that by maximizing the edge service provider's utility while considering the users' optimal responses, DisCNN enables a low-energy inference solution for EDs.

In Figs. 5(c) and 6(c), we compare the number of EDs that successfully complete inference tasks under varying numbers of EDs and models, with the model count set to 10 and the user count to 50, respectively. The results show that DisCNN significantly outperforms other baseline algorithms in terms of the number of successful inferences, with a particularly strong advantage in multi-user and multi-model scenarios. RAPJM also demonstrates commendable performance in successful task completion, achieving significantly higher success rates than traditional baselines, though still falling short of DisCNN's optimal results. This advantage stems from DisCNN's joint optimization of resource allocation, pricing, and caching, which efficiently utilizes the computational and communication resources of resource-constrained edge servers. The performance gap between RAPJM and DisCNN primarily arises because RAPJM's optimization framework focuses predominantly on minimizing end-to-end latency for individual users, without incorporating comprehensive incentive mechanisms to encourage broader ED participation. By maximizing the edge service provider's utility, DisCNN also provides latency guarantees for ED inference tasks, thereby improving ED satisfaction. Consequently, DisCNN is highly effective at delivering quality service to users in resource-constrained environments.

Figs. 5(d) and 6(d) illustrate the average inference latency under different schemes with a fixed number of models and users. When the number of models is fixed, the average inference latency of all schemes exhibits an increasing trend. However,

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

DisCNN consistently achieves the lowest average inference latency, with the slowest increase. RAPJM also demonstrates competitive performance, with latency only marginally higher than DisCNN, as its core optimization mechanism specifically targets latency reduction through dynamic partition point selection. On the other hand, when the number of users is fixed, variations in the number of models cause fluctuations in the inference latency across all schemes, but our proposed framework still achieves the lowest inference latency. Similarly, RAPJM maintains good latency performance in this scenario, though slightly exceeding DisCNN due to its lack of integrated caching and pricing considerations. Interestingly, as the number of models increases, the proposed framework demonstrates a decreasing trend in inference latency. These results strongly validate that our proposed framework, through the joint mechanisms of caching, pricing, and resource allocation, effectively reduces the average inference latency and adapts to scenarios with different scales of user numbers and inference models.

Finally, Figs. 5(e) and 6(e) present the algorithm execution time under different schemes with a fixed number of models and users. From both figures, it can be observed that as the number of users and models increases, the algorithm execution time exhibits an upward trend. However, the execution time differences across the schemes are relatively small. Notably, although the schemes combining PMC and RMC with RGC demonstrate shorter execution times, this result arises from their simplified consideration of model usability or reliance solely on constraint-satisfying strategies, which leads to reduced computational complexity. However, as indicated by previous experimental results, such simplifications result in performance that is difficult to effectively guarantee.

VII. Conclusion

In this paper, we introduced DisCNN, an end-edge collaborative inference framework for CNN in multi-access edge computing systems. Our approach addresses the limitations of storage resources and model availability on edge servers, while also considering the utility of both the service providers and the EDs. This framework is particularly suited for scenarios where a single edge server serves multiple EDs, such as in localized edge computing deployments, and can be extended to multi-edge server scenarios by integrating user allocation strategies, as discussed in related works [59], [60], [61].

Extensive simulation results demonstrate that DisCNN effectively reduces inference latency and energy consumption. It achieves a balanced optimization of caching, resource allocation, and pricing strategies, thereby maximizing the service provider's utility without compromising the performance requirements of the EDs. While the DisCNN framework is currently optimized for CNN-based inference, its core principles of game-theoretic resource allocation and GPU-based execution hold potential for broader neural network architectures, which we plan to explore in future work, alongside investigating usage-based per-unit pricing schemes to further enhance resource utilization efficiency and cost fairness for EDs.

References

[1] H. Li, X. Li, Q. Fan, Q. He, X. Wang, and V. C. Leung, "Distributed DNN inference with fine-grained model partitioning in mobile edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9060–9074, Oct. 2024.

[2] E. F. Maleki, W. Ma, L. Mashayekhy, and H. La Roche, "QoS-aware content delivery in 5G-enabled edge computing: Learning-based approaches," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9324–9336, Oct. 2024.

[3] H. Hao, C. Xu, W. Zhang, S. Yang, and G.-M. Muntean, "Joint task offloading, resource allocation, and trajectory design for multi-UAV cooperative edge computing with task priority," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8649–8663, Sep. 2024.

[4] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.

[5] J. Zhou, Q. Yang, L. Zhao, H. Dai, and F. Xiao, "Mobility-aware computation offloading in satellite edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9135–9149, Oct. 2024.

[6] H. Peng, Y. Zhan, D.-H. Zhai, X. Zhang, and Y. Xia, "Egret: Reinforcement mechanism for sequential computation offloading in edge computing," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3541–3554, Nov./Dec. 2024.

[7] X. Xu, K. Meng, H. Xiang, G. Cui, X. Xia, and W. Dou, "Blockchain-enabled secure, fair and scalable data sharing in zero-trust edge-end environment," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 2056–2069, Jun. 2025, doi: 10.1109/JSAC.2025.3560007.

[8] F. Yuan, Z. Zhang, and Z. Fang, "An effective CNN and transformer complementary network for medical image segmentation," *Pattern Recognit.*, vol. 136, 2023, Art. no. 109228.

[9] X. Yang et al., "PICO: Pipeline inference framework for versatile CNNs on diverse mobile devices," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2712–2730, Apr. 2024.

[10] T. Z. H. Ernest and A. Madhukumar, "Computation offloading in MEC-enabled IoV networks: Average energy efficiency analysis and learning-based maximization," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 6074–6087, May 2024.

[11] Z. Zhang et al., "RTCoInfer: Real-time collaborative CNN inference for stream analytics on ubiquitous images," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1212–1226, Apr. 2023.

[12] H. Shi, W. Zheng, Z. Liu, R. Ma, and H. Guan, "Automatic pipeline parallelism: A parallel inference framework for deep learning applications in 6G mobile communication systems," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 7, pp. 2041–2056, Jul. 2023.

[13] X. Xu, Y. Hu, G. Cui, L. Qi, W. Dou, and Z. Cai, "CADEC: A combinatorial auction for dynamic distributed DNN inference scheduling in edge-cloud networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10024–10041, Oct. 2025.

[14] S. Liu et al., "AdaKnife: Flexible DNN offloading for inference acceleration on heterogeneous mobile devices," *IEEE Trans. Mobile Comput.*, vol. 24, no. 2, pp. 736–748, Feb. 2025.

[15] Y. Duan and J. Wu, "Optimizing job offloading schedule for collaborative DNN inference," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3436–3451, Apr. 2024.

[16] A. Chen et al., "Opara: Exploiting operator parallelism for expediting DNN inference on GPUs," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 325–333, Jan. 2025.

[17] W. Fang, W. Xu, C. Yu, and N. N. Xiong, "Joint architecture design and workload partitioning for DNN inference on industrial IoT clusters," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–21, 2023.

[18] S. Zhang, S. Zhang, Z. Qian, J. Wu, Y. Jin, and S. Lu, "DeepSlicing: Collaborative and adaptive CNN inference with low latency," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 9, pp. 2175–2187, Sep. 2021.

[19] J. Wu, L. Wang, Q. Jin, and F. Liu, "Graft: Efficient inference serving for hybrid deep learning with SLO guarantees via DNN re-alignment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 2, pp. 280–296, Feb. 2024.

[20] G. Cui, Q. He, F. Chen, Y. Zhang, H. Jin, and Y. Yang, "Interference-aware game-theoretic device allocation for mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 4001–4012, Nov. 2022.

[21] H. Teng, Z. Li, K. Cao, S. Long, S. Guo, and A. Liu, "Game theoretical task offloading for profit maximization in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5313–5329, Sep. 2023.

[22] Z. Tong, X. Deng, J. Mei, L. Dai, K. Li, and K. Li, "Stackelberg game-based task offloading and pricing with computing capacity constraint in mobile edge computing," *J. Syst. Archit.*, vol. 137, 2023, Art. no. 102847.

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

[23] M. Datar, E. Altman, and H. L. Cadre, "Strategic resource pricing and allocation in a 5G network slicing Stackelberg game," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 502–520, Mar. 2023.
[24] K. Liu, C. Liu, G. Yan, V. C. Lee, and J. Cao, "Accelerating DNN inference with reliability guarantee in vehicular edge computing," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3238–3253, Dec. 2023.
[25] L. Yang, C. Zheng, X. Shen, and G. Xie, "OfpCNN: On-demand fine-grained partitioning for CNN inference acceleration in heterogeneous devices," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3090–3103, Dec. 2023.
[26] Q. Chen et al., "Towards real-time inference offloading with distributed edge computing: The framework and algorithms," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7552–7571, Jul. 2024.
[27] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: A distributed online approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8129–8142, Aug. 2024.
[28] C.-W. Shao, Y.-F. Li, C.-Y. Shen, S.-Z. Liu, and Z. Yang, "Risk-sharing mechanism design in non-cooperative multi-defender Stackelberg defense resources allocation game," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 6653–6665, Oct. 2024.
[29] X. Huang et al., "Service reservation and pricing for green metaverses: A Stackelberg game approach," *IEEE Wireless Commun.*, vol. 30, no. 5, pp. 86–94, Oct. 2023.
[30] L. Xie, S. Meng, W. Yao, and X. Zhang, "Differential pricing strategies for bandwidth allocation with LFA resilience: A Stackelberg game approach," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4899–4914, 2023.
[31] J. Li, W. Liang, Y. Li, Z. Xu, X. Jia, and S. Guo, "Throughput maximization of delay-aware DNN inference in edge computing by exploring DNN model partitioning and inference parallelism," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 3017–3030, May 2023.
[32] Y. Hu, X. Xu, L. Duan, M. Bilal, Q. Wang, and W. Dou, "End-edge collaborative inference of convolutional fuzzy neural networks for big data-driven Internet of Things," *IEEE Trans. Fuzzy Syst.*, vol. 33, no. 1, pp. 203–217, Jan. 2025.
[33] J. Kim, Y. Park, G. Kim, and S. J. Hwang, "SplitNet: Learning to semantically split deep networks for parameter reduction and model parallelization," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1866–1874.
[34] P. Dai, B. Han, K. Li, X. Xu, H. Xing, and K. Liu, "Joint optimization of device placement and model partitioning for cooperative DNN inference in heterogeneous edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 210–226, Jan. 2025.
[35] Z. Wei, B. Li, R. Zhang, X. Cheng, and L. Yang, "Many-to-many task offloading in vehicular fog computing: A multi-agent deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2107–2122, Mar. 2024.
[36] X. Wang, D. Wu, X. Wang, R. Zeng, L. Ma, and R. Yu, "Truthful auction-based resource allocation mechanisms with flexible task offloading in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 6377–6391, May 2024.
[37] Q. Xiao, G. Li, Z. Liu, and A. Hu, "Optimal subcarrier allocation scheme for physical-layer key generation in an OFDMA network," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4929–4942, 2025.
[38] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
[39] X. Chen, J. Zhang, B. Lin, Z. Chen, K. Wolter, and G. Min, "Energy-efficient offloading for DNN-based smart IoT systems in cloud-edge environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 683–697, Mar. 2022.
[40] Z. Liu et al., "DNN partitioning, task offloading, and resource allocation in dynamic vehicular networks: A Lyapunov-guided diffusion-based reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1945–1962, Mar. 2025.
[41] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, "Supply and threshold voltage scaling for low power CMOS," *IEEE J. Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, Aug. 1997.
[42] G. Zhang, W. Zhang, Y. Cao, D. Li, and L. Wang, "Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4642–4655, Oct. 2018.
[43] Z. Sun, G. Sun, Y. Liu, J. Wang, and D. Cao, "BARGAIN-MATCH: A game theoretical approach for resource allocation and task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1655–1673, Feb. 2024.
[44] S. R. Ghorpade and B. V. Limaye, *A Course in Multivariable Calculus and Analysis*. Berlin, Germany: Springer, 2010.
[45] W. Yu, K. Yang, Y. Bai, T. Xiao, H. Yao, and Y. Rui, "Visualizing and comparing AlexNet and VGG using deconvolutional layers," in *Proc. 33rd Int. Conf. Mach. Learn. Workshop Visualization Deep Learn.*, New York, NY, USA, Jun. 2016. [Online]. Available: https://icmlviz.github.io/icmlviz2016/assets/papers/4.pdf
[46] D. Theckedath and R. Sedamkar, "Detecting affect states using VGG16, ResNet50 and SE-ResNet50 networks," *SN Comput. Sci.*, vol. 1, no. 2, 2020, Art. no. 79.
[47] S. Targ, D. Almeida, and K. Lyman, "ResNet in ResNet: Generalizing residual architectures," 2016, *arXiv:1603.08029*.
[48] P. Ballester and R. Araujo, "On the performance of GoogLeNet and AlexNet applied to sketches," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 1124–1128.
[49] Y. Zhu and S. Newsam, "DenseNet for dense flow," in *Proc. IEEE Int. Conf. Image Process.*, 2017, pp. 790–794.
[50] X. Xia, C. Xu, and B. Nan, "Inception-v3 for flower classification," in *Proc. 2nd Int. Conf. Image Vis. Comput.*, 2017, pp. 783–787.
[51] U. Kulkarni, S. Meena, S. V. Gurlahosur, and G. Bhogar, "Quantization friendly MobileNet (QF-MobileNet) architecture for vision based applications on embedded platforms," *Neural Netw.*, vol. 136, pp. 28–39, 2021.
[52] S. An, M. Lee, S. Park, H. Yang, and J. So, "An ensemble of simple convolutional neural network models for MNIST digit recognition," 2020, *arXiv: 2008.10400*.
[53] S. Jošilo and G. Dán, "Wireless and computing resource allocation for selfish computation offloading in edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 2467–2475.
[54] F. Tütüncüoğlu and G. Dán, "Optimal pricing for service caching and task offloading in edge computing," in *Proc. 17th Wireless On-Demand Netw. Syst. Serv. Conf.*, 2022, pp. 1–8.
[55] Z. Shao, L. T. Landau, and R. C. de Lamare, "Dynamic oversampling for 1-bit ADCs in large-scale multiple-antenna systems," *IEEE Trans. Commun.*, vol. 69, no. 5, pp. 3423–3435, May 2021.
[56] A. A. Bian, J. M. Buhmann, A. Krause, and S. Tschiatschek, "Guarantees for greedy maximization of non-submodular functions with applications," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 498–507.
[57] X. Wang, R. Li, C. Wang, X. Li, T. Taleb, and V. C. Leung, "Attention-weighted federated deep reinforcement learning for device-to-device assisted heterogeneous collaborative edge caching," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 154–169, Jan. 2021.
[58] Y. Li et al., "Real-time adaptive partition and resource allocation for multi-user end-cloud inference collaboration in mobile environment," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13076–13094, Dec. 2024.
[59] Y.-J. Liu et al., "Trusted clustering based federated learning in edge networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9726–9742, Oct. 2025.
[60] Z. Ding, D. Kong, Z. Zhang, X. Xie, and J. Xu, "ClusPar: A game-theoretic approach for efficient and scalable streaming edge partitioning," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 116–130, Jan. 2025.
[61] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.

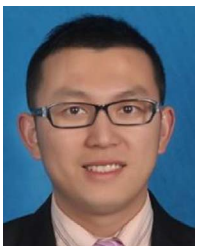
Maowen Li received the BEng degree in software engineering from the Nanjing University of Information Science & Technology, China, in June 2024. He is currently working toward the postgraduate degree with the School of Software Engineering, Nanjing University of Information Science & Technology. His research interests include edge computing and collaborative inference.

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.

Xiaolong Xu (Senior Member, IEEE) received the PhD degree in computer science and technology from Nanjing University, China, in 2016. He is currently a full professor with the School of Software, Nanjing University of Information Science and Technology. He has published more than 100 peer-review articles in international journals and conferences, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Fuzzy Systems*, *IEEE Transactions on Intelligent Transportation Systems*, IJCAI, ICDM, ICWS, ICSOC, etc. He was selected as the highly cited researcher of Clarivate (2021–2024). He received best paper awards from Tsinghua Science and Technology at 2023, the *Journal of Network and Computer Applications* at 2022, and several conferences, including IEEE HPCC 2023, IEEE ISPA 2022, IEEE CyberSciTech 2021, IEEE CPSCom2020, etc. His research interests include edge computing, the Internet of Things (IoT), cloud computing, and Big Data.

Guangming Cui received the master's degree from Anhui University, China, in 2018, and the PhD degree from the Swinburne University of Technology, Australia, in 2022, in computer science. Currently, he is an associate professor with the Nanjing University of Information Science & Technology, China. He has published more than 30 peer-review articles in international journals and conferences, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, ICWS, ICSOC, etc. His research interests include edge computing, service computing, mobile computing, and software engineering.

Yong Cheng received the PhD degree in computer science and technology from Wuhan University, China, in 2009. He is currently a full professor with the School of Software, Nanjing University of Information Science and Technology. He has published some articles in international journals, including the *IEEE Internet of Things Journal*, etc. His research interests include Big Data, Internet of Things, and deep learning.



Fei Dai received the PhD degree from Yunnan University, Kunming, China, in 2011. He is currently a full professor with the School of Big Data and Intelligence Engineering, Southwest Forestry University, Kunming, China. His research interests include edge computing, business process management, and software engineering. He has authored and co-authored more than 80 peer review papers in international journals and conferences including *IEEE Transactions on Industrial Informatics*, *IEEE Internet of Things Journal*, *World Wide Web Journal*, *Software: Practice and Experience*, IEEE International Conference on Web Services, etc. He is the recipient of the Best Paper Award of IEEE International Conference on Cloudcomp 2019.

Wanchun Dou received the PhD degree in mechanical and electronic engineering from the Nanjing University of Science and Technology, China, in 2001. He is currently a full professor with the State Key Laboratory for Novel Software Technology, Nanjing University. From April 2005 to June 2005 and from November 2008 to February 2009, he visited the Departments of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong, respectively, as a visiting scholar. He has published more than 100 research papers in international journals and international conferences. His research interests include workflow, cloud computing, and service computing.

Lianyong Qi (Senior Member, IEEE) received the PhD degree from the Department of Computer Science and Technology, Nanjing University, China, in 2011. He is currently a full professor with the College of Computer Science and Technology, China University of Petroleum (East China), China. He has already published more than 100 articles, including *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Big Data*, *Future Generation Computer Systems*, *Journal of Computational Social Science*, CCPE, ICWS, and ICSOC. His research interests include services computing, Big Data, and the Internet of Things.

Zhipeng Cai (Fellow, IEEE) received the BS degree from the Beijing Institute of Technology, Beijing, China, in 2001, and the MS and PhD degrees from the Department of Computing Science, University of Alberta, Edmonton, AB, Canada, in 2004 and 2008, respectively. He is currently a professor with the Department of Computer Science, Georgia State University, Atlanta, GA, USA. His research has received funding from multiple academic and industrial sponsors, including the National Science Foundation and the U.S. Department of State, and has resulted in more than 100 publications in top journals and conferences, with more than 14500 citations, including more than 80 IEEE/ACM transactions papers. His research expertise lies in the areas of resource management and scheduling, privacy, networking, and Big Data.

Authorized licensed use limited to: Reykjavik University. Downloaded on March 12,2026 at 11:09:09 UTC from IEEE Xplore. Restrictions apply.